University of Liberal Arts Bangladesh

Department of Computer Science & Engineering

CSE 2201 : Algorithms

Lecture 12:
Greedy Problems – Bin Packing

Atanu Shuvam Roy

Lecturer,
Dept. of CSE, ULAB, Dhaka

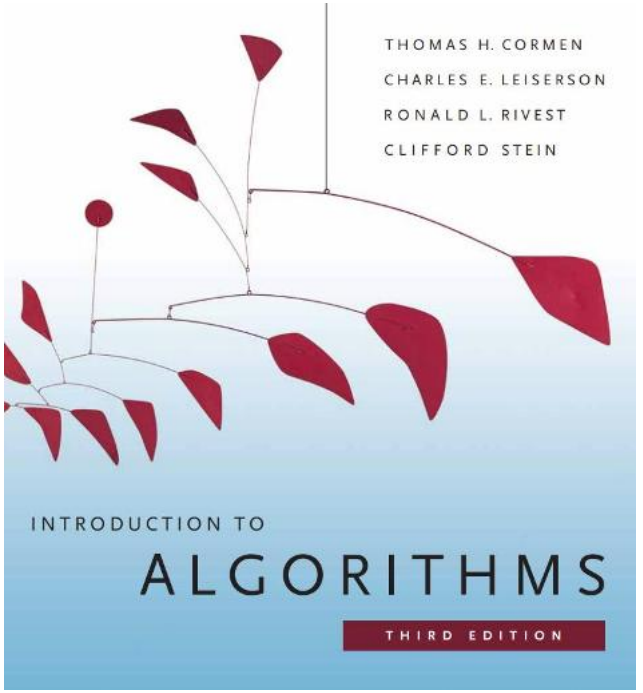
EMAIL:
atanu.shuvam@ulab.edu.bd

Class Code
GVWTHYJV

Fall 2025

Reference Book

- Introduction to Algorithms, 3rd edition. T.H. Cormen, C.E. Leiserson, R.L. Rivest, C. Stein



Distribution of Marks

Assessments	%
Mid Term Examination	25
Final Examination	40
Attendance & class performance	10
Assignment & Quizzes	25
Total	100

Today's Contents

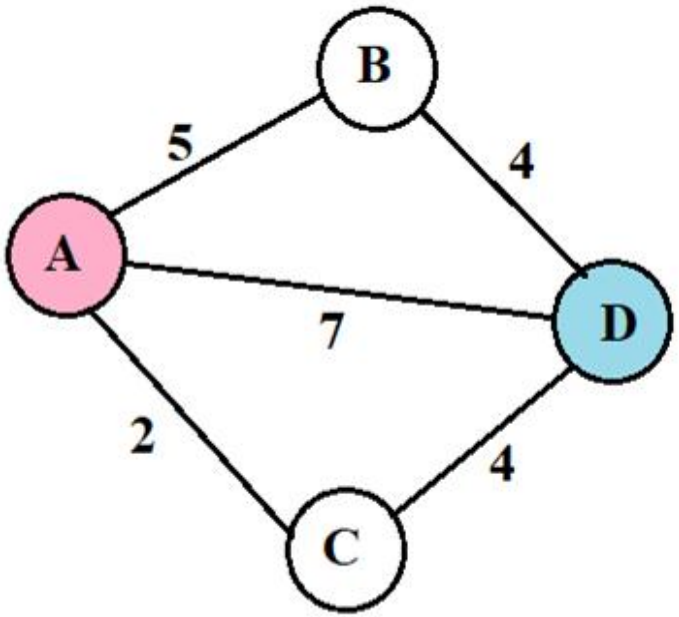
- Introduction to Greedy Approach
- Discussion on Coin Change Problem
- Discussion on Bin packing Problem

Greedy Algorithm

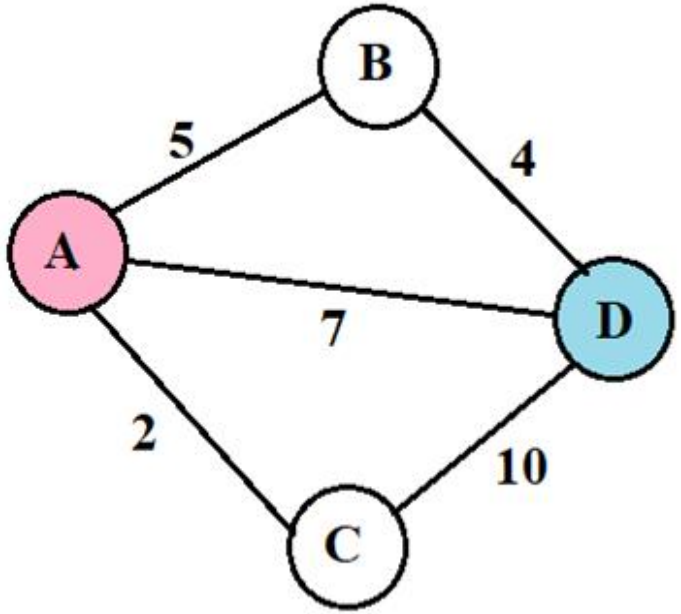
- Greedy algorithms make the choice that **looks best at the moment**.
- Find **simple** and **straightforward** solutions.
- Gives you **locally optimal** solution **without looking ahead**.
- This locally optimal choice **may or may not lead to a globally optimal solution** (i.e. an optimal solution to the entire problem).

Example

- Find the Shortest Path From a Graph **(Node A -> D)**



A -> D : A -> C -> D = 2 + 4 =6



A ->D : A -> C -> D = 2 + 10 =12

Advantages and disadvantages

- **Advantages**
  - **Simple**
  - **Quick**
  - **Easy to program**

- **Disadvantages**
  - **Does not always lead to a global optimal solution.**

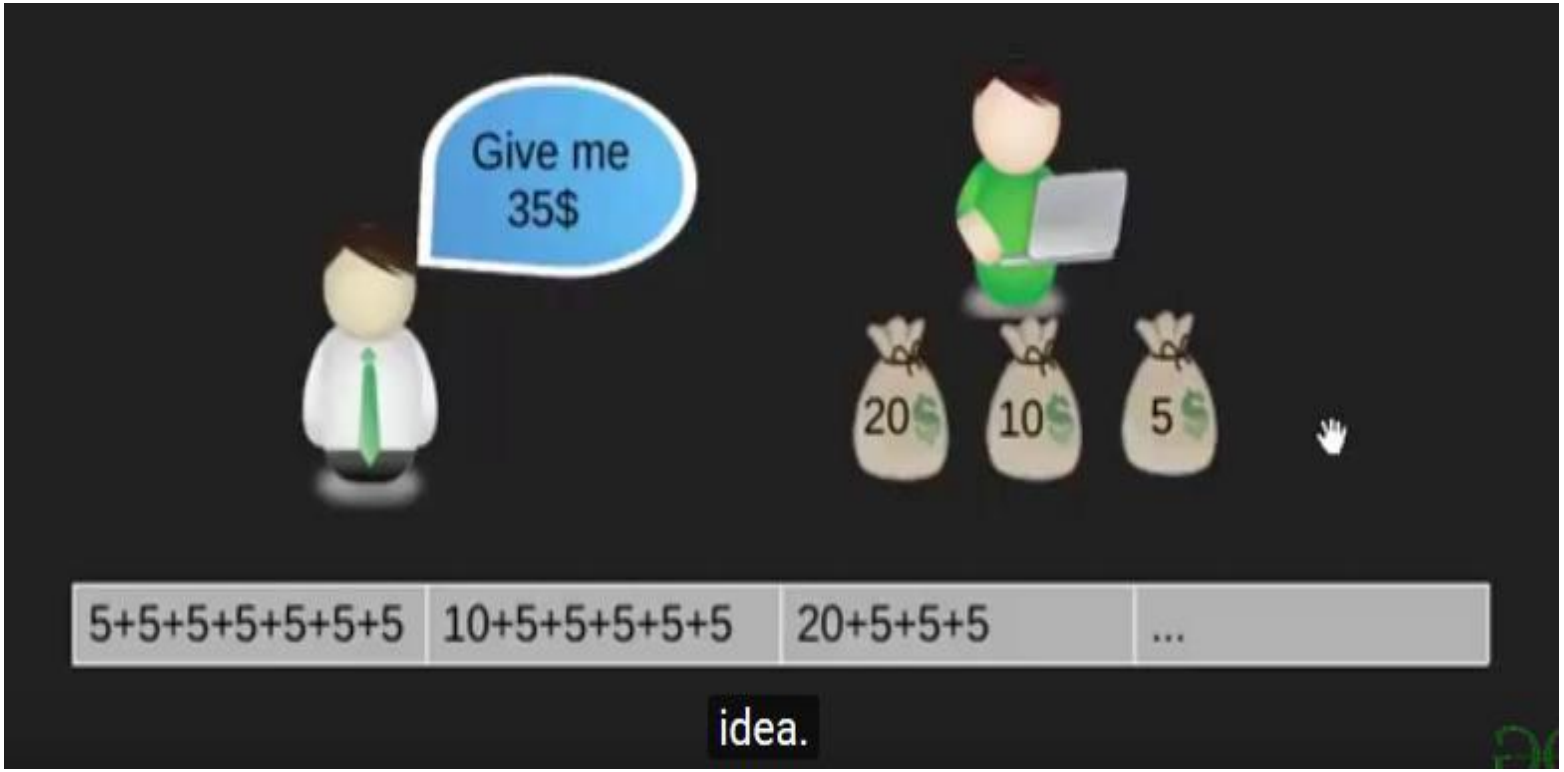
We are going to learn

1. Greedy Coin Change
2. Greedy Bin Packing
3. Greedy Partial Knapsack
4. Greedy Huffman Coding

Greedy Algorithms

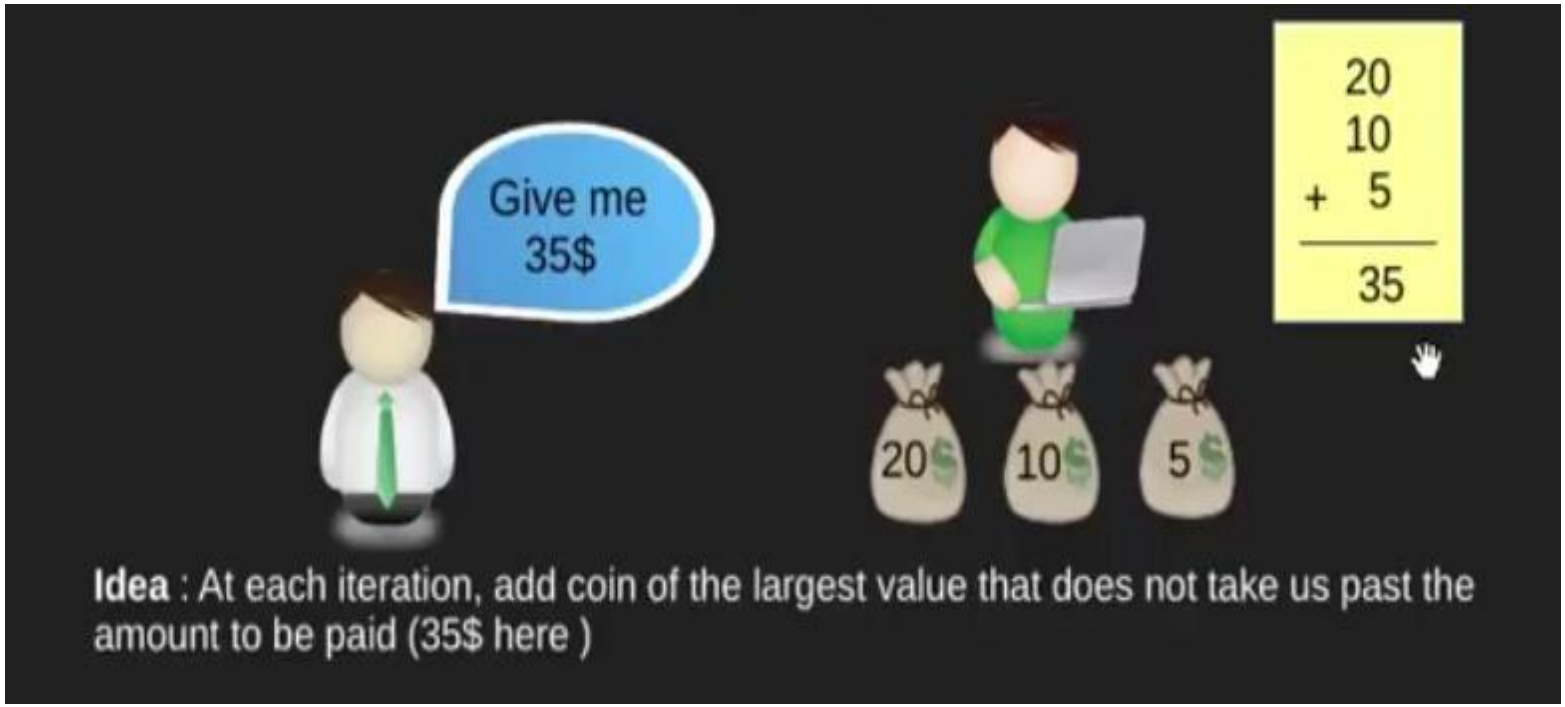
Part -1 : Greedy Coin Changing Algorithm

Coin Change



Objective of Coin Change

Pay out a given sum *S* with the smallest number of coins possible.



Coin Change

- Assume that we have an **unlimited** number of coins of various denominations:

$1, $5, $10, $25, $50

- Now use a greedy method to give the **least amount of coins** for **$41**

Coin Change

- Assume that we have an unlimited number of coins of various denominations:

$1, $5, $10, $25, $50

- Now use a greedy method to give the least amount of coins for **$41**

41 – 25 = 16 ------ 25
16 – 10 = 6 ------- 10
6 – 5 = 1 ------- 5
1 - 1 = 0 ------- 1

We have to give: 25 10 5 1

Making Change – A big problem - 1

- Assume that we have an unlimited number of coins of various denominations:

  $4 , $10, $25

- Now use a greedy method to give the least amount of coins for **$41**

  - **» 41 – 25 = 16 ------ 25**
  - **» 16 – 10 = 6 ------- 10**
  - **» 6 – 4 = 2 ------- 4**
  - **» 2 ------- ???**

We should choose 25 4 4 4 4

Making Change – A big problem -2

- Coins are valued

$30, $20, $5, $1

Make change for $40

–Does not have greedy-choice property, since **$40** is best made with two **$.20's**,

–but the greedy solution will pick **three coins** (**which ones?**)

The greedy coin changing Algorithm (Bad Example)

```
coins = [20, 10, 5, 2, 1]   // assume sorted descending
S = amount to change

while S > 0 do
   // Find the largest coin <= S using a loop
   c = 0
   for i = 0 to length(coins)-1 do
      if coins[i] <= S then
         c = coins[i]
         break
      end if
   end for

   // Count how many coins of value c we can use
   num = 0
   for j = 1 to (S / c) do
      num = num + 1
   end for

   // "Pay out" num coins
   print "Use", num, "coin(s) of", c

   // Update remaining amount
   S = S - num * c
end while
```

Make change for 45

20	10	5	2	1

The greedy coin changing Algorithm (Good Example)

Input: S = 45
Coins = [20, 10, 5, 2, 1]

```
for each coin c in Coins do
    if S == 0 then
        break
    end if

    num = S // c

    if num > 0 then
        print("Use", num, "coin(s) of", c)
        S = S - num * c
    end if
end for
```

Make change for 45

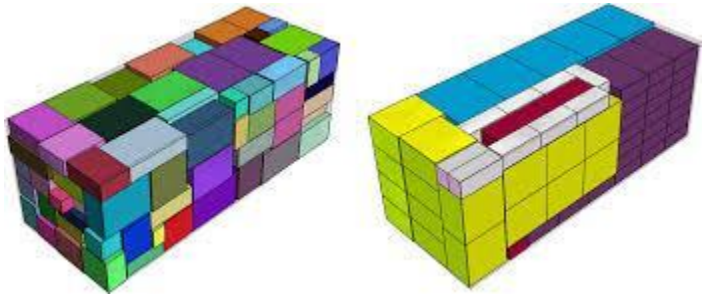
20	10	5	2	1

Step-by-Step Execution

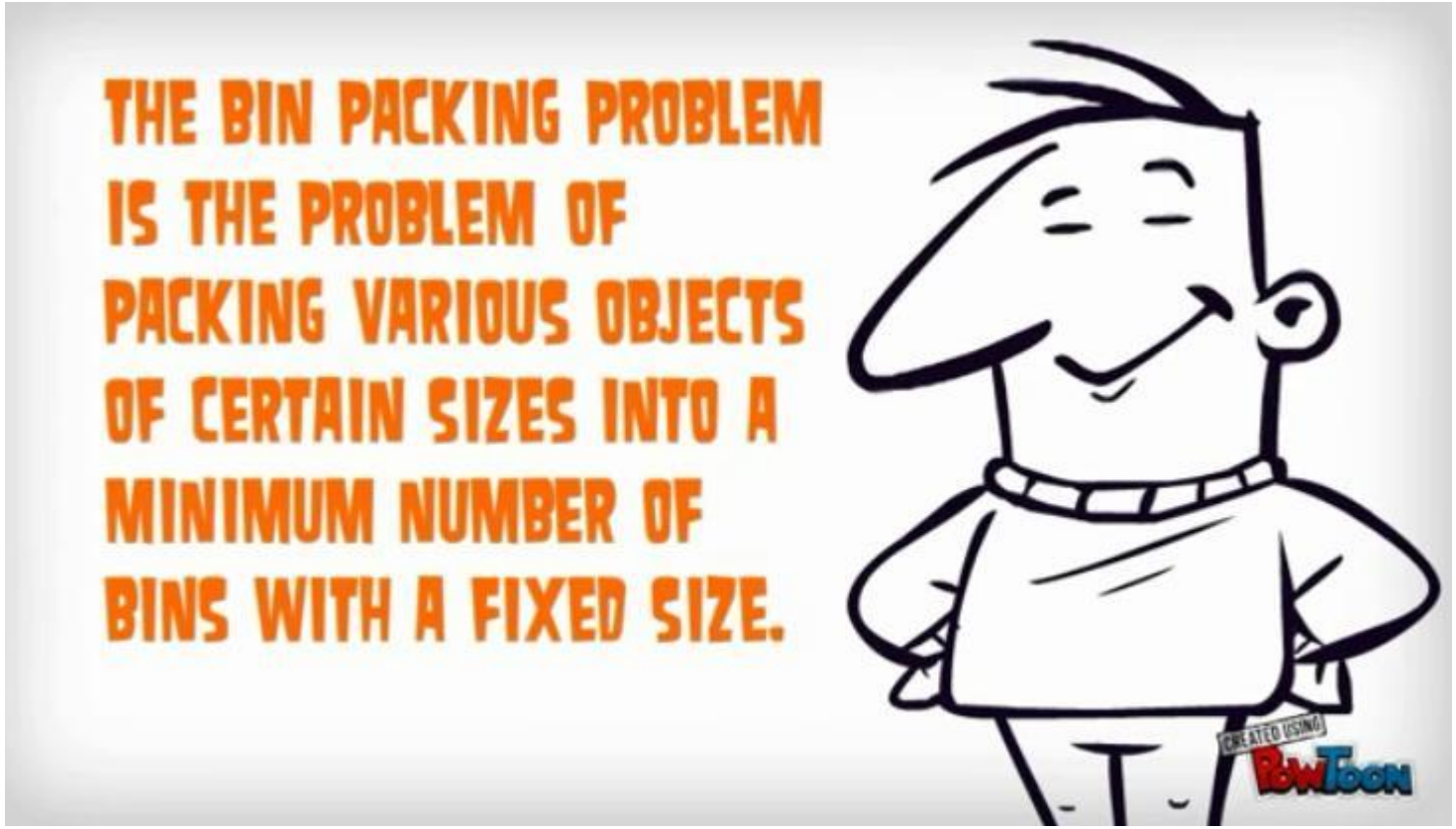
- **20-cent coin**
  - num = 45 // 20 = 2
  - Use 2 coins of 20 → subtract 2*20 = 40
  - Remaining S = 45 - 40 = 5
- **10-cent coin**
  - num = 5 // 10 = 0
  - Skip
- **5-cent coin**
  - num = 5 // 5 = 1
  - Use 1 coin of 5 → subtract 1*5 = 5
  - Remaining S = 5 - 5 = 0
- **2-cent coin** and **1-cent coin**
  - Remaining S = 0, stop

Greedy Algorithms

Part -2 : Bin Packing Problem



Concept of Bin Packing



Bin Packing

- In the **bin packing problem**, objects of different volumes must be packed into a **finite** number of **bins or containers** each of volume *V* in a way that **minimizes** the number of bins used.

- There are many **variations** of this problem,

  such as 2D packing,

  linear packing,

  packing by weight,

  packing by cost, and so on.

- They have many applications, such as **filling up containers, loading trucks with weight capacity constraints, creating file backups in media** and so on.

Ways of Solution

- First Fit Algorithm
- First Fit Decreasing Algorithm
- Full Bin Algorithm
- Other Algorithms

Find The Lower Bound

Total size =

12 + 12 + 7 + 6 + 10 + 4 + 5 + 1 = 57

Bin (Ship) Size = 20

Lower Bound = ⌈ 57/20 ⌉

= ⌈2.85⌉ = 3

1. First Fit Algorithm

- This is a very **straightforward** greedy approximation algorithm.
- The algorithm processes the items in **arbitrary** order.
- For each item, it attempts to place the item in the first bin that can accommodate the item. If no bin is found, it opens a new bin and puts the item within the new bin.

How First Fit Algorithm works

First Fit Solution

pgsql

Copy code

```
Input: items[] = array of item sizes
       bin_capacity = maximum capacity of a bin
Output: bins[] = list of bins with assigned items

bins = empty list

for each item in items[] do
    placed = false
    for each bin in bins do
        if bin.remaining_capacity >= item then
            bin.add(item)
            placed = true
            break
        end if
    end for
    if not placed then
        new_bin = create bin with capacity bin_capacity
        new_bin.add(item)
        bins.append(new_bin)
    end if
end for
```

2. First Fit Decreasing Algorithm

- **Sort the items in decreasing order.**
- **Repeat the process *First Fit*.**

How First Fit Decreasing works

First Fit Decreasing Solution

3. Full Bin Algorithm (Strategy)

- The full bin packing algorithm is more likely to produce an **optimal solution** – using the least possible number of bins – than the first fit decreasing and first fit algorithms.

- **It works by matching object so as to fill as many bins as possible.**

Full Bin Concept

- Place the Items into the most full bin, which could accept it.
- Keeps bins open even when the next item in the list will not fit in the previous opened bins, in the hope that a later smaller item will fill.
- Put it in the bins so that smallest empty space is left.

How Full Bin Algorithm Works

Full Bin Solution

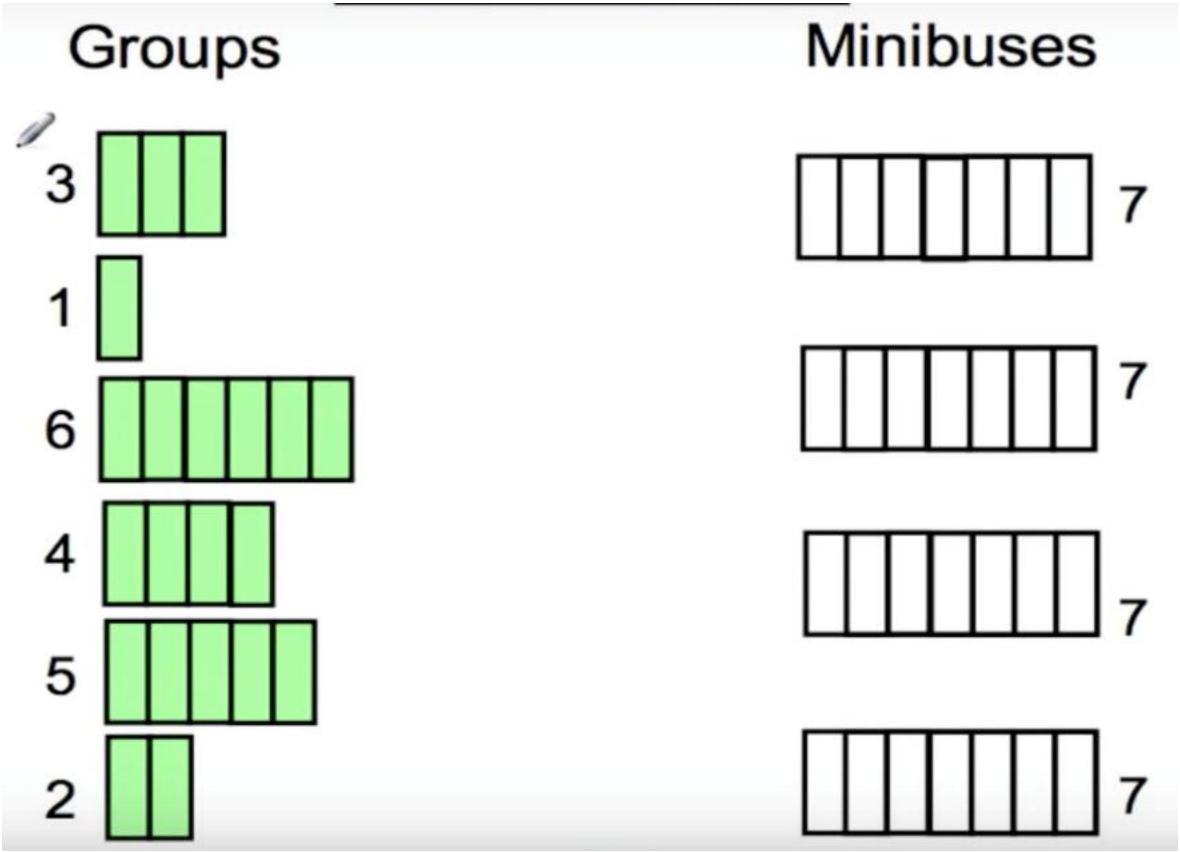
Analysis of Three Algorithms

Algorithm	Features
• **FIRST FIT**	▪ Easiest to use ▪ Isn't optimal
• **FIRST FIT DCREASING**	▪ Easy to use ▪ Isn't optimal always
• **FULL BIN**	▪ Optimal ▪ Difficult to use

Exercise- 1

6 groups of people, of group sizes 3, 1, 6, 4, 5 and 2 need to fit onto minibuses with capacity 7 but must stay together in their groups.
Find the number of minibuses need to pack them in efficiently and so that each group stays together.

Exercise- 1

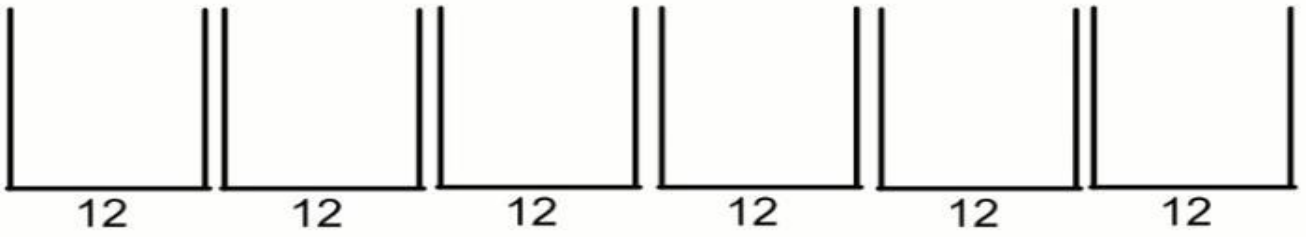


Exercise- 2

1. Find the lower bound.
2. Find how many bins are required to move these items using three algorithms.
3. Which is more efficient and why?

BIN PACKING LOWER BOUND

A(2) B(7) C(3) D(3) E(2) F(3)
G(4) H(6) I(7) J(4) K(3) L(4)



Reference

- http://coursera.org/learn/algorithms-part1/
- GeeksForGeeks
- YouTube
- LLMs
- Old Lectures of ULAB (Pramanik Sir)

Thank You

atanu.Shuvam@ulab.edu.bd

University of Liberal Arts Bangladesh

Department of Computer Science & Engineering